



UNIVERSIDADE PAULISTA

CST EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

PROJETO INTEGRADO MULTIDISCIPLINAR

Desenvolvimento de um Sistema Acadêmico Colaborativo com Apoio de IA.

GIOVANI RODRIGUES DE A. REIS

KLEITON FACHETTI

KEMELLY MARIA FERNANDES LIMA

NICOLLY LIMA SOARES

PEDRO HENRIQUE DE PAULA SANTOS

VITOR DE SOUSA V. AREIAS

SÃO PAULO - SP

2025

GIOVANI RODRIGUES DE A. REIS

KLEITON FACHETTI

KEMELLY MARIA FERNANDES LIMA

NICOLLY LIMA SOARES

PEDRO HENRIQUE DE PAULA SANTOS

VITOR DE SOUSA V. AREIAS

PROJETO INTEGRADO MULTIDISCIPLINAR

Desenvolvimento de um Sistema Acadêmico Colaborativo com Apoio de IA.

Projeto Integrado Multidisciplinar para
obtenção do título de tecnólogo em Análise
e Desenvolvimento de Sistemas,
apresentado à Universidade Paulista –
UNIP.

Orientador: Prof. Denilson Souza.

SÃO PAULO - SP

2025

GIOVANI RODRIGUES DE A. REIS
KLEITON FACHETTI
KEMELLY MARIA FERNANDES LIMA
NICOLLY LIMA SOARES
PEDRO HENRIQUE DE PAULA SANTOS
VITOR DE SOUZA V. AREIS

PROJETO INTEGRADO MULTIDISCIPLINAR

Desenvolvimento de um Sistema Acadêmico Colaborativo com Apoio de IA.

Trabalho de conclusão de curso para
obtenção do título de graduação em (nome
do curso) apresentado à Universidade
Paulista – UNIP.

Aprovado(a) em: ____/____/____

BANCA EXAMINADORA

Prof. ou Profa. Dr(a)./Me(a). xxxxxxxxxxxxx
Universidade Paulista - UNIP

Prof. ou Profa. Dr(a)./ Me(a). xxxxxxxxxxxxx
Universidade Paulista - UNIP

Prof. ou Profa. Dr(a)./ Me(a). xxxxxxxxxxxxx
Universidade Paulista - UNIP

RESUMO

O presente trabalho tem como objetivo o desenvolvimento de um **Sistema Acadêmico Colaborativo com Apoio de Inteligência Artificial (IA)**, voltado à modernização da gestão educacional e à integração entre alunos, professores e secretaria. A proposta surge da necessidade de centralizar informações acadêmicas e reduzir o uso de processos manuais, substituindo planilhas e registros físicos por uma plataforma digital automatizada, acessível e sustentável. Para isso, foram aplicadas metodologias ágeis, com destaque para o **Scrum**, que possibilitou o acompanhamento iterativo do projeto e a colaboração entre os membros da equipe. A implementação técnica foi realizada em **Python**, utilizando o **framework FastAPI** para o backend e uma estrutura modular, segura e escalável. O sistema contempla funcionalidades como cadastro e gerenciamento de turmas, controle de notas, geração automática de relatórios e atendimento automatizado via **chatbot com IA**, que fornece respostas em tempo real a dúvidas frequentes. O uso de **diagramas UML** e **modelagem de rede cliente-servidor** assegurou clareza arquitetural e padronização técnica. Os resultados demonstram que o sistema alcança plenamente os objetivos propostos, proporcionando maior eficiência na comunicação institucional e promovendo práticas de sustentabilidade digital. O projeto também favoreceu o desenvolvimento de competências técnicas e interpessoais dos integrantes, consolidando a interdisciplinaridade entre as disciplinas do curso. Conclui-se que o sistema proposto é uma solução tecnológica inovadora e escalável, capaz de transformar o ambiente acadêmico em um espaço mais colaborativo, automatizado e sustentável, em consonância com as demandas contemporâneas da transformação digital na educação.

Palavras-chave: Inteligência Artificial. Sistema Acadêmico. Metodologia Ágil. FastAPI. Sustentabilidade Digital.

ABSTRACT

This project aims to develop an **Academic Collaborative System with Artificial Intelligence (AI) Support**, designed to modernize educational management and improve integration among students, teachers, and the academic administration. The proposal arises from the need to centralize academic information and replace manual and fragmented processes with an automated, digital, and sustainable platform. Agile methodologies, particularly **Scrum**, were applied to ensure efficient teamwork, iterative delivery, and transparent task management. The system was developed in **Python**, using the **FastAPI framework** to build a secure, modular, and scalable backend. The platform includes key features such as class and student management, grade control, automatic report generation, and an **AI-powered chatbot** capable of responding to frequent inquiries in real time. The adoption of **UML diagrams** and a **client-server network model** provided structural clarity and technical consistency. The results demonstrate that the system fully meets its objectives, optimizing institutional communication while promoting environmental responsibility through paperless operations. Furthermore, the project contributed to the academic and professional growth of the team, fostering technical proficiency, collaboration, and problem-solving skills. In conclusion, the proposed system stands out as an innovative and scalable technological solution that transforms the academic environment into a more collaborative, intelligent, and sustainable ecosystem, aligned with the current challenges of digital transformation in education.

Keywords: Artificial Intelligence. Academic System. Agile Methodology. FastAPI. Digital Sustainability.

LISTA DE ILUSTRAÇÕES

Figura 1 – Diagrama de Caso de Uso do Sistema.....	12
Figura 2 – Diagrama de Classes do Sistema.....	13
Figura 3 – Diagrama de Sequência do Sistema.....	14
Figura 4 – Trecho de Autenticação de Usuários.....	17
Figura 5 – Trecho de Consulta/Cadastro dos Alunos.....	18
Figura 6 – Trecho de Criação/Gerenciamento das Turmas.....	19
Figura 7 – Trecho de Controle de Atividades.....	20
Figura 8 – Trecho de Perguntas e Respostas com ChatBot.....	21
Figura 9 – Trecho de Coleta de Notas.....	22
Figura 10 – Diagrama de Rede do Projeto.....	30
Figura 11 – Acesso do Aluno no Sistema.....	33
Figura 12 – Acesso do Professor(a) no Sistema.....	34
Figura 13 – Acesso Consulta Rápida e Funções do Professor(a).....	34
Figura 14 – ChatBot do Professor(a).....	35
Figura 15 – ChatBot do Aluno.....	35

LISTA DE ABREVIATURAS

AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CST	Curso Superior de Tecnologia
CSS	Cascading Style Sheets
FAQ	Frequently Asked Questions
FastAPI	Framework Python para desenvolvimento de APIs rápidas e assíncronas
GitHub	Plataforma de hospedagem e controle de versões de código-fonte
IP	Internet Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LAN	Local Area Network
ML	Machine Learning
NLP	Natural Language Processing
PIM	Projeto Integrado Multidisciplinar
PO	Product Owner
Scrum	Framework Ágil de Gerenciamento e Desenvolvimento de Projetos
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language
UNIP	Universidade Paulista
URL	Uniform Resource Locator
UX	User Experience
VM	Virtual Machine
VENV	Virtual Environment

LISTA DE TABELAS

Tabela 1 – Síntese da Aplicação dos Artefatos.....	24
Tabela 2 – Eventos do Scrum e Ciclo Semanal.....	25

SUMÁRIO

RESUMO.....	4
ABSTRACT	5
LISTA DE ILUSTRAÇÕES	6
LISTA DE ABREVIATURAS.....	7
LISTA DE TABELAS	8
SUMÁRIO.....	9
1 INTRODUÇÃO	10
2 LINGUAGEM UNIFICADA DE MODELAGEM (UML)	11
2.1 Aplicação no Sistema Acadêmico Colaborativo com IA.....	11
2.2 Importância da UML em um Sistema com IA.....	15
2.3 Relação com a Inteligência Artificial.....	16
3 CÓDIGO FONTE FUNCIONAL E DOCUMENTADO	17
3.1 Login e Autenticação.....	17
3.2 Cadastro e Consulta de Alunos	18
3.3 Criação e Gerenciamento de Turmas	19
3.4 Controle de Atividades e Notas.....	20
3.5 Chatbot de Dúvidas Frequentes	21
3.6 Geração de Relatórios	21
4 USO DE METODOLOGIA ÁGIL.....	23
4.1 Conceito de SCRUM e sua Aplicação.....	23
4.2 Ferramentas de Apoio: JIRA e Organização das Tarefas	24
4.3 Eventos do SCRUM e Ciclo Semanal	25
4.4 Papéis e Responsabilidades.....	25
5 DIAGRAMA DE REDE DE COMPUTADORES.....	29
6 MANUAL DE USO DO SISTEMA	31
6.1 Eficácia do Manual.....	32
6.2 Manual do Sistema Garante	32
7 ARQUITETURA E ESTRUTURA TÉCNICA.....	36
8 TECNOLOGIAS EMERGENTES	38
9 CONSIDERAÇÕES FINAIS	41
10 CONCLUSÃO	43
REFERÊNCIAS.....	45

1 INTRODUÇÃO

O presente projeto tem como objetivo o desenvolvimento de um Sistema Acadêmico Colaborativo (PIM), uma plataforma digital segura voltada à gestão acadêmica integrada e à otimização da comunicação entre alunos, professores e secretaria. O sistema busca unificar o controle de turmas, atividades, notas e relatórios, oferecendo um ambiente interativo e acessível, com o apoio de recursos de Inteligência Artificial (IA) para o atendimento inteligente por meio de chatbots.

A proposta surgiu da necessidade de otimizar a gestão acadêmica e reduzir o uso de planilhas e comunicações fragmentadas por e-mails ou aplicativos de mensagens. Dessa forma, propõe-se uma solução tecnológica colaborativa. O sistema foi projetado para operar em ambiente cliente-servidor, permitindo que múltiplos usuários acessem simultaneamente as funcionalidades em rede local. O projeto também contempla práticas de engenharia de software ágil, integrando metodologias de desenvolvimento iterativo, documentação técnica e testes contínuos.

Para o desenvolvimento da solução, será adotada a metodologia ágil de engenharia de software, com o uso de sprints, backlog e acompanhamento contínuo do progresso do projeto. As implementações envolverão o uso de Python, explorando algoritmos e estruturas de dados para busca, ordenação e geração de relatórios e Programação Estruturada em C. Também serão elaborados diagramas UML para representar casos de uso, classes e sequências, garantindo uma modelagem clara e consistente do sistema. Além de conceitos complementares de Redes de Computadores e Sistemas Distribuídos. A Inteligência Artificial será explorada principalmente como um recurso de apoio à personalização e automação de processos dentro do sistema com módulos de chatbot destinados a diferentes perfis de usuários.

Além de seu valor técnico, o projeto busca contribuir para a sustentabilidade por meio da substituição de processos impressos por soluções digitais, reduzindo o consumo de papel e incentivando práticas acadêmicas mais ecológicas. O resultado final é uma plataforma funcional, intuitiva e colaborativa, capaz de melhorar a comunicação e o fluxo de informações dentro do ambiente educacional.

2 LINGUAGEM UNIFICADA DE MODELAGEM (UML)

A *Unified Modeling Language* (UML) é uma linguagem padronizada utilizada na **engenharia de software** para modelar, visualizar e documentar sistemas de forma estruturada e compreensível. Criada pela *Object Management Group* (OMG), desenvolvida para unificar e padronizar notações de modelagem orientada a objetos. A UML oferece uma forma universal de representar graficamente os elementos de um sistema, desde seus componentes técnicos até as interações entre usuários e processos.

Na UML, os diagramas são organizados em categorias que refletem diferentes perspectivas do sistema. A distinção fundamental é entre duas categorias principais:

- **Diagramas estruturais:** apresentam a arquitetura estática do sistema — ou seja, mostram os componentes, as classes, os objetos, os pacotes, as interfaces, e as relações entre eles. Acesse: [Miro, guia de diagramas](#).
- **Diagramas comportamentais:** ilustram os aspectos dinâmicos do sistema — como ele reage a interações, mudanças de estado, fluxos de atividades e trocas de mensagens entre os componentes. Acesse: [Visual Paradigm, guia](#).

Por exemplo:

- Um *Diagrama de Classes* (estrutura) mostra as classes, atributos, métodos e as relações de herança ou associação.
- Um *Diagrama de Caso de Uso* ou *Diagrama de Sequência* (comportamento) mostra como os atores interagem com o sistema ou como os objetos trocam mensagens ao longo do tempo.

Essa distinção ajuda a cobrir tanto o “esqueleto” do sistema (estrutura) quanto o “funcionamento” do sistema (comportamento) de forma complementar.

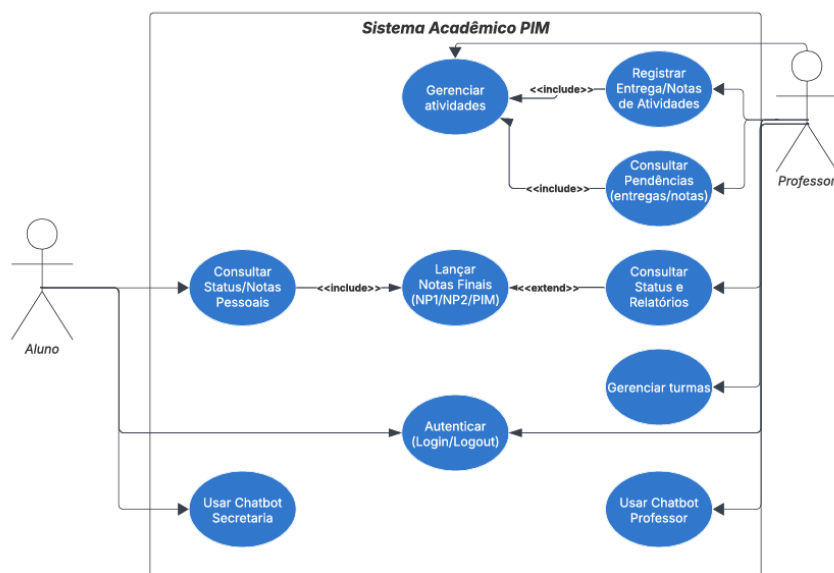
2.1 APLICAÇÃO NO SISTEMA ACADÊMICO COLABORATIVO COM IA

No contexto do sistema acadêmico colaborativo com apoio de Inteligência Artificial, a utilização da Unified Modeling Language (UML) permite estruturar, clarificar e documentar como os diferentes módulos interagem — envolvendo tanto atores humanos (alunos, professores, secretaria) quanto componentes técnicos (interface web, base de dados, Chatbot de IA) — garantindo alinhamento entre todos os envolvidos.

Diagrama de Caso de Uso:

Este diagrama concentra-se em representar as **interações essenciais** entre os atores externos ao sistema e o sistema propriamente dito. Demonstra como o sistema responde a diferentes perfis de usuário. Essa visão de alto nível permite compreender o escopo funcional do sistema, definir requisitos e orientar o desenvolvimento dos módulos, dando uma visão de alto nível do que o sistema ou parte dele faz, sem entrar em detalhes de implementação.

Figura 1 – Diagrama de Caso de Uso do Sistema



Fonte: Autores

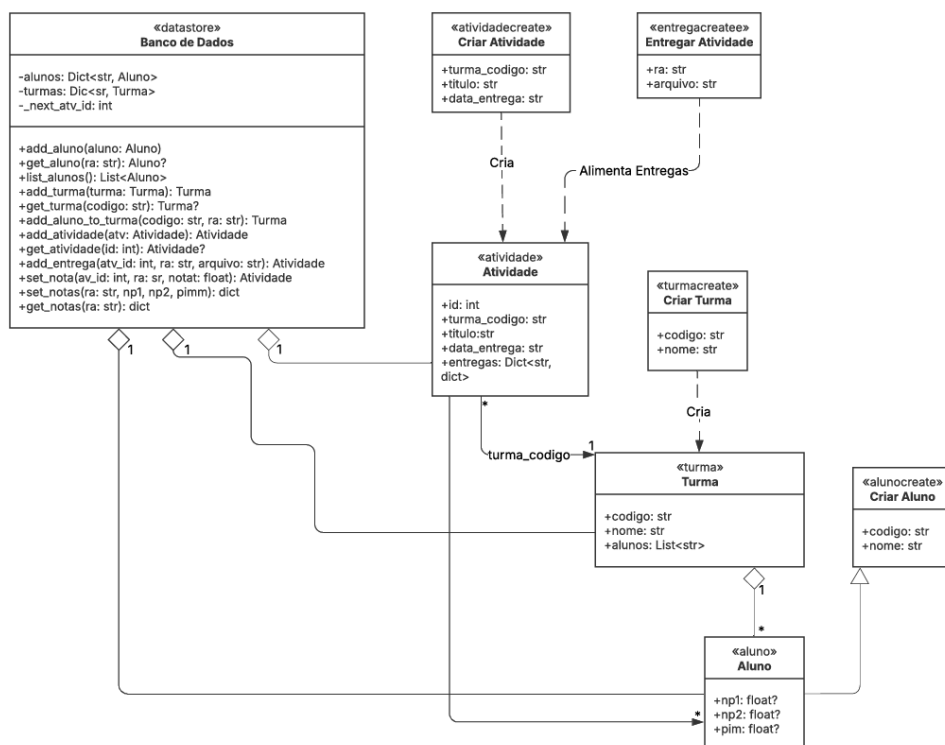
No diagrama de caso de uso, ocorre uma representação simplificada, porém eficaz das funcionalidades oferecidas pelo sistema, mostrando quais atores (como aluno, professor ou secretaria) interagem com ele e de que forma. Nele, cada ação relevante

(por exemplo, “Consultar status acadêmico” ou “Lançar notas”) é apresentada como um “uso” que está dentro da fronteira do sistema, enquanto os atores situam-se fora dessa fronteira, apontando suas relações com os casos de uso. Essa visualização permite às partes interessadas — sejam desenvolvedores, usuários ou gestores — compreenderem rapidamente **o que** o sistema faz, **quem** o utiliza e **como** esses atores se relacionam com as funcionalidades principais, sem necessariamente mergulhar nos detalhes de implementação.

Diagrama de Classes:

Aqui aborda-se a **estrutura lógica estática** do sistema: classes, atributos, métodos, e as relações entre essas classes (associação, herança, composição). No projeto, existem classes como Aluno, Professor, Turma, Atividade, que se conectam a um datastore central para gerenciar dados e validações de notas. Este diagrama serve como base para a codificação em linguagens como Python e para a documentação da arquitetura. Descreve a estrutura de um sistema mostrando as classes, seus atributos, operações e as relações entre elas.

Figura 2 – Diagrama de Classes do Sistema



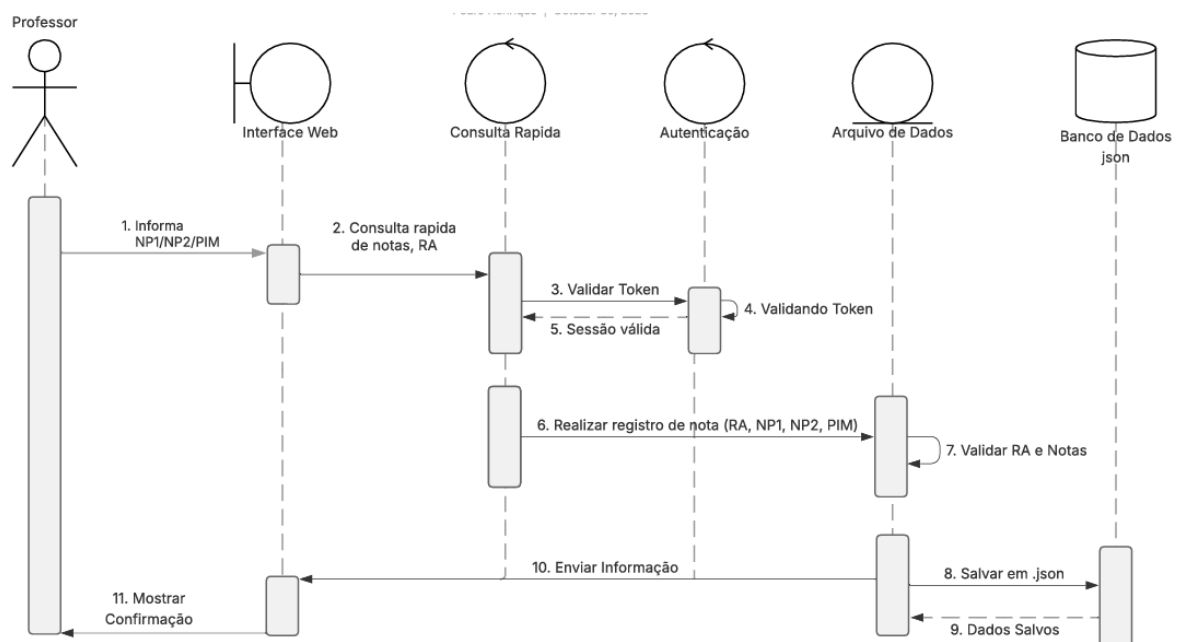
Fonte: Autores

O diagrama de classes permite identificar claramente os papéis de cada elemento dentro do sistema — como uma classe “Aluno” com atributos (matrícula, nome, status) e métodos consultarNotas(), atualizarDados(), ou uma classe “Turma” com métodos de gerenciamento de alunos — e como essas classes se relacionam, por exemplo, um professor instrui várias turmas, ou uma turma contém vários alunos. Esse tipo de representação facilita a transição entre design e implementação e torna mais transparente o impacto de alterações na estrutura do sistema.

Diagrama de Sequência:

Este diagrama foca na **ordem cronológica** das interações entre objetos ou componentes, desde a interface até o banco de dados, no cumprimento de determinado caso de uso. No sistema em questão, ele descreve o fluxo completo de operações: autenticação, lançamento de notas, validação de tokens, entre outros. Esse tipo de diagrama é fundamental para garantir consistência nas trocas de mensagens e para visualização clara do comportamento dinâmico do sistema. Representam como os objetos interagem entre si ao longo do tempo.

Figura 3 – Diagrama de Sequência do Sistema



Fonte: Autores

Nesse tipo de diagrama, cada participante (objeto, ator ou módulo) é representado por uma **lifeline** (linha vertical) que indica seu período de atividade, e as setas horizontais mostram as mensagens enviadas e recebidas entre esses participantes, ordenadas de cima para baixo conforme o tempo avança. Além disso, são utilizadas notações para identificar chamadas síncronas ou assíncronas, retorno de mensagens, criação ou destruição de objetos, bem como fragmentos de interação que expressam condições alternativas (como “if/else”), loops ou paralelismos. Dessa forma, o diagrama de sequência permite mapear com precisão como o sistema responde a uma solicitação (por exemplo, autenticação de usuário ou lançamento de nota), mostrando passo a passo qual objeto chama qual método, em qual ordem, e qual objeto realiza cada tarefa — o que auxilia tanto no entendimento do comportamento dinâmico quanto no alinhamento da implementação com o planejamento arquitetural.

Os diagramas da Unified Modeling Language (UML) revelam-se como alicerces sólidos para o desenvolvimento do sistema acadêmico colaborativo com IA fornecendo uma visão estruturada, clara e integrada das interações dos usuários, da arquitetura interna e do comportamento dinâmico da aplicação. A sua relevância transcende o simples ato de documentar: eles atuam como guias para o processo de desenvolvimento, asseguram a coesão entre os módulos de IA e o restante do sistema, e favorecem uma manutenção sistemática e eficaz ao longo do ciclo de vida do software. Nesse contexto, os diagramas UML não são apenas artefatos de apoio, mas elementos estratégicos em projetos de engenharia de software — sejam eles de natureza acadêmica ou profissional.

2.2 IMPORTÂNCIA DA UML EM UM SISTEMA COM IA

A adoção de UML é essencial para sistemas acadêmicos com IA pelos seguintes motivos:

Padronização e comunicação eficiente;

Permite que equipes multidisciplinares (programadores, analistas e especialistas em IA) compreendam o sistema por meio de representações gráficas universais.

Integração com Inteligência Artificial;

Diagramas ajudam a mapear os pontos de interação entre o sistema tradicional e os módulos de IA (como o Chatbot de suporte descrito no manual com acesso no [GitHub](#)).

Manutenção e escalabilidade;

Ao representar relações entre classes e processos, a UML facilita futuras expansões — como novas funcionalidades de aprendizado de máquina.

Automação de testes e simulações;

Os diagramas servem de base para scripts automatizados, verificando coerência entre a lógica projetada e o código real.

Apoio à documentação técnica e acadêmica;

No contexto do projeto, os diagramas UML cumprem papel essencial na validação do sistema e na entrega de documentação profissional.

2.3 RELAÇÃO COM A INTELIGÊNCIA ARTIFICIAL

A IA atua como **módulo colaborativo** que interpreta intenções e gera respostas automáticas aos usuários (alunos e professores). O uso da UML possibilita:

- **Mapear o fluxo cognitivo da IA** — desde a entrada do texto até a resposta processada;
- **Identificar dependências lógicas** entre o Chatbot, o banco de dados e o módulo de notas;
- **Garantir coerência entre processos humanos e automatizados**, permitindo que a IA opere de forma alinhada ao sistema principal.

3 CÓDIGO FONTE FUNCIONAL E DOCUMENTADO

O código do sistema foi desenvolvido com a linguagem Python, utilizando o framework FastAPI para o backend e integração com a interface web. O sistema é dividido em módulos que organizam suas principais funções:

- Login e autenticação: garantindo o acesso do aluno e do professor, que possuem níveis de acesso diferentes.
- Gerenciamento de turmas e alunos: cadastrar, editar e consultar informações.
- Controle de notas: realiza o cálculo automático das médias (NP1, NP2 e PIM).
- Chatbot: para responder dúvidas frequentes (FAQs)
- Relatórios: gera informação sobre o desempenho dos alunos.

O código-fonte do **sistema** encontra-se hospedado no repositório oficial do projeto, disponível no [Github](#)

3.1 LOGIN E AUTENTIFICAÇÃO

O trecho de código abaixo apresenta a estrutura de autenticação do sistema, responsável por gerenciar o acesso de alunos e professores, garantindo níveis de permissão diferentes.

No exemplo, observa-se que os dados dos usuários são armazenados em um dicionário (USERS), com informações como nome de usuário, senha, papel (role) e, no caso dos alunos, o número de registro acadêmico (ra).

Figura 4 – Trecho de Autenticação de Usuários

```
22 # -----
23 # "Banco" de usuários fixos para demo
24 # - username + { password, role, ra(opcional), username }
25 # -----
26 ✓ USERS: Dict[str, Dict] = {
27     "aluno": {
28         "username": "aluno",
29         "password": "aluno123",
30         "role": "aluno",
31         # RA associado ao aluno (usado pelo /aluno/status)
32         "ra": "H76DJH0",
33     },
34     "professor": {
35         "username": "professor",
36         "password": "prof123",
37         "role": "professor",
38     },
39 }
```

Fonte: Autores

Esse código é utilizado pelo módulo de autenticação (auth.py) para validar as credenciais enviadas na rota /auth/login. Dependendo do papel do usuário, o sistema concede acesso apenas às rotas correspondentes — por exemplo, somente professores podem cadastrar ou editar informações de alunos e turmas.

3.2 CADASTRO E CONSULTA DE ALUNOS

O módulo alunos.py concentra as rotas responsáveis pelo gerenciamento dos alunos.

O código abaixo mostra como é feito o cadastro de novos alunos e a listagem dos já existentes, sempre com a segurança de acesso garantida pela dependência “require_professor”, que limita essas ações aos professores.

Figura 5 – Trecho de Consulta/Cadastro dos Alunos

```

19 # -----
20 # CRUD simples de alunos (mantenha os seus já existentes)
21 # -----
22 @router.post("/", response_model=Aluno)
23 def criar_aluno(payload: AlunoCreate, _=Depends(require_professor)):
24     try:
25         return db.add_aluno(Aluno(**payload.model_dump()))
26     except ValueError as e:
27         raise HTTPException(status_code=400, detail=str(e))
28
29 @router.get("/", response_model=list[Aluno])
30 def listar_alunos(_=Depends(require_professor)):
31     return db.list_alunos()
32
33 @router.get("/{ra}", response_model=Aluno)
34 def obter_aluno(ra: str, _=Depends(require_professor)):
35     aluno = db.get_aluno(ra)
36     if not aluno:
37         raise HTTPException(status_code=404, detail="Aluno não encontrado.")
38     return aluno
39

```

Fonte: Autores

Essas funções fazem a ponte entre o front-end e o banco de dados, permitindo cadastrar alunos, editar dados e consultar notas.

O professor tem acesso total às informações, enquanto o aluno só pode consultar seus próprios resultados.

3.3 CRIAÇÃO E GERENCIAMENTO DE TURMAS

A parte de gerenciamento de turmas está centralizada no arquivo `turmas.py`.

O trecho abaixo mostra a função que cria uma nova turma, permitindo que o professor defina o código e o nome da turma, com a lista de alunos iniciando vazia.

Figura 6 – Trecho de Criação/Gerenciamento das Turmas

```

22 # Cria o agrupador de rotas para "turmas"
23 router = APIRouter(prefix="/turmas", tags=["turmas"])
24
25 # -----
26 # POST /turmas/ → Criar nova turma
27 # -----
28 @router.post("/", response_model=Turma, dependencies=[Depends(require_professor)])
29 def criar_turma(payload: TurmaCreate):
30     """
31     Cria uma turma nova com código e nome.
32     - payload: { "codigo": "TADS01", "nome": "Análise e Desenvolvimento..." }
33     - alunos[] inicia vazio por padrão.
34     """
35     try:
36         # Monta objeto Turma com lista de alunos vazia
37         turma = Turma(codigo=payload.codigo, nome=payload.nome, alunos=[])
38         # Persiste via repositório (também grava no JSON)
39         return db.add_turma(turma)
40     except ValueError as e:
41         # Ex.: turma duplicada (código já existente)
42         raise HTTPException(status_code=400, detail=str(e))
43
44 # -----
45 # GET /turmas/{codigo} → Buscar detalhes da turma
46 # -----
47 @router.get("/{codigo}", response_model=Turma, dependencies=[Depends(require_professor)])
48 def obter_turma(codigo: str):
49     """
50     Retorna dados completos de uma turma (código, nome e RAs dos alunos).
51     """
52     turma = db.get_turma(codigo)
53     if not turma:
54         raise HTTPException(status_code=404, detail="Turma não encontrada.")
55     return turma
56
57 # -----
58 # POST /turmas/{codigo}/alunos/{ra} → Adicionar aluno em turma
59 # -----
60 @router.post("/{codigo}/alunos/{ra}", response_model=Turma, dependencies=[Depends(require_professor)])
61 def adicionar_aluno_na_turma(codigo: str, ra: str):
62     """
63     Inclui o RA de um aluno já cadastrado dentro da turma.
64     - Valida se a turma existe e se o RA do aluno existe.
65     - Evita duplicidade de RA na lista.
66     """
67     try:
68         return db.add_aluno_to_turma(codigo, ra)
69     except ValueError as e:
70         # Mensagens amigáveis do repositório: "Turma não encontrada", "Aluno (RA) não encontrado" etc.
71         raise HTTPException(status_code=400, detail=str(e))
72

```

Fonte: Autores

Esse recurso é essencial para organizar os alunos em grupos, facilitando o controle de notas e a atribuição de atividades. O uso da dependência

“require_professor” reforça a segurança do sistema, garantindo que apenas docentes possam realizar essa ação.

3.4 CONTROLE DE ATIVIDADES E NOTAS

O módulo atividades.py gerencia as tarefas e avaliações criadas pelos professores. O exemplo a seguir mostra a função responsável por criar uma atividade associada a uma turma já existente no sistema.

Figura 7 – Trecho de Controle de Atividades

```

26 # -----
27 # POST /atividades/ → Criar nova atividade
28 # -----
29 @router.post("/", response_model=Atividade, dependencies=[Depends(require_professor)])
30 def criar_atividade(payload: AtividadeCreate):
31     """
32     Cria atividade vinculada a uma turma existente.
33     - payload: { "turma_codigo": "TAD001", "titulo": "Atv 1", "data_entrega": "2025-11-30" }
34     - O ID é atribuído automaticamente no repositório.
35     - Inicia com entregas vazias.
36     """
37     turma = db.get_turma(payload.turma_codigo)
38     if not turma:
39         raise HTTPException(status_code=404, detail="Turma não encontrada.")
40
41     # Monta objeto Atividade; id será substituído no add_atividade()
42     atv = Atividade(
43         id=0,
44         turma_codigo=payload.turma_codigo,
45         titulo=payload.titulo,
46         data_entrega=payload.data_entrega,
47         entregas={}
48     )
49     return db.add_atividade(atv)
50
51 # -----
52 # GET /atividades/{atividade_id} → Obter atividade específica
53 # -----
54 @router.get("/{atividade_id}", response_model=Atividade, dependencies=[Depends(require_professor)])
55 def obter_atividade(atividade_id: int):
56     """
57     Retorna todos os dados de uma atividade.
58     - Se não existir, retorna 404.
59     """
60     atv = db.get_atividade(atividade_id)
61     if not atv:
62         raise HTTPException(status_code=404, detail="Atividade não encontrada.")
63     return atv

```

Fonte: Autores

Essa lógica impede que uma atividade seja criada em uma turma inexistente, assegurando a integridade dos dados.

Posteriormente, o professor pode lançar notas para cada aluno matriculado nessa turma, calculando automaticamente as médias ponderadas (NP1, NP2 e PIM).

3.5 CHATBOT DE DÚVIDAS FREQUENTES

O chatbot (localizado em chatbot.py) serve como uma ferramenta de autoatendimento, permitindo que alunos consultem respostas para dúvidas comuns, como questões sobre carteirinha, provas substitutivas e prazos de matrícula.

Figura 8 – Trecho de Perguntas e Respostas com ChatBot

```

21  FAQ = [
22      {
23          "key": "provas_substitutivas",
24          "pergunta": "Perdi uma prova, como funciona as substitutivas?",
25          "resposta": (
26              "Provas substitutivas: agende no portal do aluno e compareça "
27              "na semana de provas substitutivas, no dia do professor da disciplina. "
28              "Neste ano, de 21/novembro a 24/novembro."
29          ),
30      },
31      {
32          "key": "carteirinha_2via",
33          "pergunta": "Como solicito 2ª via da carteirinha?",
34          "resposta": (
35              "2ª via da carteirinha: solicite no portal do aluno (Serviços → Documentos). "
36              "Retirada na secretaria em até 5 dias úteis."
37          ),
38      },
39      {
40          "key": "trancamento",
41          "pergunta": "Como funciona trancamento de disciplina?",
42          "resposta": (
43              "Trancamento de disciplina: abra requerimento no portal até a 4ª semana letiva. "
44              "Sujeito a cobrança proporcional conforme regulamento."
45          ),
46      },
47      {
48          "key": "revisao_nota",
49          "pergunta": "Como pedir revisão de nota?",
50          "resposta": (
51              "Revisão de nota: solicite no portal até 48 horas após a publicação da nota. "
52              "A coordenação responde em até 7 dias."
53          ),
54      },
55  ]
56

```

Fonte: Autores

Essas informações ficam armazenadas em uma lista de dicionários chamada FAQ, que é facilmente atualizada pelos administradores.

O chatbot retorna as respostas por meio das rotas configuradas na API, servindo como uma interface automatizada de atendimento estudantil.

3.6 GERAÇÃO DE RELATÓRIOS

O módulo relatorios.py é responsável por reunir e processar dados das turmas e atividades, gerando estatísticas que ajudam o professor a acompanhar o progresso dos alunos.

O trecho a seguir demonstra a função que coleta as notas de cada atividade para compor os relatórios de desempenho.

Figura 9 – Trecho de Coleta de Notas

```

24 # -----
25 # Função utilitária: coletar notas válidas (0..10) de uma atividade
26 # -----
27 def _coletar_notas_da_atividade(atv) -> List[float]:
28     """
29     Coleta todas as notas válidas (0..10) de uma atividade.
30     - As entregas podem estar como dict (com chave "nota") ou como objetos
31       Pydantic com atributo "nota". Esta função trata ambos os casos.
32     - Ignora ausências de nota ou valores inválidos.
33     """
34     notas: List[float] = []
35     for payload in atv.entregas.values():
36         # Caso 1: dicionário {"arquivo": "...", "nota": 8.5}
37         if isinstance(payload, dict):
38             valor = payload.get("nota", None)
39         else:
40             # Caso 2: objeto com atributo "nota" (robustez)
41             valor = getattr(payload, "nota", None)
42
43         if valor is None:
44             continue
45
46         try:
47             n = float(valor)
48         except (TypeError, ValueError):
49             continue
50
51         if 0.0 <= n <= 10.0:
52             notas.append(n)
53
54     return notas

```

Fonte: Autores

Essa função percorre todas as entregas registradas e filtra apenas as notas válidas, servindo de base para o cálculo de médias e para os gráficos de desempenho exibidos nos relatórios.

Com isso, o sistema fornece uma visão analítica da performance geral dos alunos e turmas.

O sistema acadêmico do projeto integra de forma eficiente backend em Python com FastAPI e interface web, oferecendo módulos para autenticação, gerenciamento de alunos e turmas, controle de notas, chatbot e geração de relatórios. Cada funcionalidade foi desenvolvida para garantir segurança, integridade dos dados e permissões adequadas a alunos e professores, ao mesmo tempo que facilita o cálculo automático de médias e a análise de desempenho. Dessa forma, o sistema proporciona uma gestão acadêmica prática, confiável e escalável, atendendo às necessidades de usuários e administradores de maneira eficiente.

4 USO DE METODOLOGIA ÁGIL

Diante da complexidade do projeto e da necessidade de integração entre diferentes áreas como programação, modelagem UML, relatórios e interface, o uso de uma metodologia ágil se mostrou essencial para garantir **organização, colaboração e entregas contínuas de valor**. Assim, o **Scrum** foi adotado como framework principal de gerenciamento, além de favorecer a entrega de partes funcionais do sistema de maneira incremental, **Scrum** oferece adaptabilidade e comunicação eficaz apoiado pela ferramenta **Atlassian Jira**. Utilizada como suporte para o gerenciamento das atividades e acompanhamento das sprints, o **Atlassian Jira** torna-se uma ferramenta de apoio viável permitindo que o grupo planejasse, executasse e avaliasse o progresso de forma estruturada e visual.

4.1 CONCEITO DE SCRUM E SUA APLICAÇÃO

O **Scrum** é uma metodologia ágil de gerenciamento de projetos voltada para **entregas incrementais e iterativas**. Seu foco está na **colaboração entre os membros da equipe**, na **adaptação rápida a mudanças** e na **entrega contínua de partes funcionais do produto**. Organiza o desenvolvimento de projetos em ciclos curtos denominados **Sprints**, geralmente com duração de uma a quatro semanas. Cada sprint resulta em uma entrega funcional do produto, chamada **incremento**, que agrega valor e é revisada antes do início do próximo ciclo.

Essa metodologia é estruturada sobre três pilares:

- **Transparência**, onde todas as etapas e atividades são visíveis para a equipe;
- **Inspeção**, que consiste na análise constante do progresso;
- **Adaptação**, que permite ajustes no planejamento sempre que necessário.

No projeto, o Scrum foi aplicado em ciclos semanais chamados **Sprints**, nos quais cada integrante tinha tarefas definidas, reuniões de acompanhamento e entregas específicas.

Nos relatórios semanais do time, disponível em: [GitHub](#), as atividades incluíram desde **estudos e criação de diagramas UML** até o **desenvolvimento da interface em Python** e **implementação de funcionalidades do sistema acadêmico**, é possível observar a aplicação prática desses pilares. O time se reunia semanalmente para definir tarefas, discutir dificuldades e planejar as próximas atividades — o que reflete fielmente o ciclo iterativo e colaborativo proposto pelo Scrum.

4.2 FERRAMENTAS DE APOIO: JIRA E ORGANIZAÇÃO DAS TAREFAS

Para a aplicação prática da metodologia ágil **Scrum**, a equipe utilizou a ferramenta **Atlassian Jira Software** como principal meio de **planejamento, acompanhamento e controle das atividades**. O Jira foi essencial para estruturar o projeto conforme os artefatos do Scrum — **Product Backlog, Sprint Backlog e Incrementos** —, além de oferecer **transparência e rastreabilidade** a todas as etapas do desenvolvimento. O uso do Jira permitiu que a equipe visualizasse o progresso em tempo real, atribuísse responsabilidades a cada integrante e organizasse as tarefas de forma dinâmica e colaborativa, disponível em: [GitHub](#)

As principais áreas e recursos utilizados dentro da ferramenta foram:

Tabela 1 – Síntese da Aplicação dos Artefatos

Artefato Scrum	Aplicação no Projeto PIM	Exemplo Prático no Jira	Resultado
Product Backlog	Repositório de todas as funcionalidades e requisitos do sistema	Tarefas como PIM-30, PIM-34, PIM-35, PIM-38	Organização e priorização das entregas
Sprint Backlog	Planejamento semanal das tarefas da sprint ativa	PIM-16, PIM-28, PIM-31	Execução iterativa e controle de progresso
Incremento	Entregas funcionais testadas e integradas ao sistema	Módulos de alunos, relatórios e chatbots	Sistema acadêmico completo e funcional

Fonte: Autores

4.3 EVENTOS DO SCRUM E CICLO SEMANAL

O Scrum define um conjunto de eventos recorrentes que estruturam o trabalho da equipe:

Tabela 2 - Eventos do Scrum e Ciclo Semanal

<i>Evento</i>	Exemplo no Projeto	Objetivo
<i>Sprint Planning</i>	Semana 1 (28/08/2025): definição de atividades como estudo de diagramas UML e Python	Planejar o conteúdo da sprint
<i>Daily Scrum</i>	Reuniões semanais relatadas (“Relatório Semanal – Projeto PIM”)	Compartilhar progresso e dificuldades diárias
<i>Sprint Review</i>	Semana 5 (16/10/2025): “Manual de Uso do Sistema apresentado e aprovado pelo grupo”	Revisar entregas e validar incrementos
<i>Sprint Retrospective</i>	Comentários reflexivos dos membros (“Foi ótimo aprender diagramas...”, “Preciso me organizar melhor”)	Analisar o processo e propor melhorias

Fonte: Autores

Esses eventos garantiram uma **rotina de inspeção e adaptação contínua**, contribuindo diretamente para o amadurecimento técnico e organizacional da equipe.

4.4 PAPÉIS E RESPONSABILIDADES

No framework Scrum, existem três papéis fundamentais que estruturam o trabalho e garantem o bom andamento do projeto: **Product Owner (PO)**, **Scrum Master** e **Time de Desenvolvimento**. Cada função possui responsabilidades específicas, mas todas se complementam para assegurar o alinhamento, a qualidade e a entrega contínua de valor.

No contexto do Projeto, **esses papéis foram adaptados e distribuídos entre os integrantes da equipe**, de modo a refletir a natureza colaborativa do projeto e as diferentes competências técnicas e organizacionais de cada membro.

Product Owner (PO):

O papel de **Product Owner** foi exercido de forma compartilhada entre o **professor orientador** e os integrantes, que atuou como ponto de contato direto entre o time e a orientação acadêmica.

O PO foi responsável por:

- **Definir e priorizar as funcionalidades** do sistema acadêmico colaborativo;
- **Aprovar as entregas** realizadas a cada sprint, garantindo o alinhamento com os objetivos do projeto;
- **Atualizar o backlog** no Jira, ordenando tarefas conforme o valor de entrega e o feedback recebido;
- **Orientar as decisões de escopo**, como a inclusão dos **chatbots inteligentes** para professores e secretaria e a divisão de perfis de usuários (aluno, professor e administrador).

Durante a **Sprint 5** disponível no [GitHub](#), o PO participou da reunião registrada como *PIM-37 – Relatório de Reunião com o Professor Orientador*, momento em que revisou o andamento do projeto, sugeriu melhorias na interface, novas funcionalidades e validou a proposta de integração da **inteligência artificial** como apoio funcional no sistema.

Scrum Master:

A função de **Scrum Master** foi desempenhada pelo integrante **VITOR DE SOUSA V. AREIAS**, que assumiu a responsabilidade de manter o time alinhado aos princípios ágeis e facilitar a execução das sprints.

Suas principais atribuições incluíram:

- **Coordenar as reuniões semanais** e assegurar que todos os integrantes estivessem cientes de suas tarefas;
- **Identificar e remover impedimentos**, como dificuldades técnicas nos diagramas de sequência e organização de código em Python;
- **Registrar o progresso no Jira**, garantindo visibilidade das tarefas e atualizações em tempo real;
- **Fomentar a colaboração** e o cumprimento dos prazos definidos no cronograma das sprints.

Time de Desenvolvimento:

O **Time de Desenvolvimento** foi composto por um grupo multifuncional, cujas habilidades complementares permitiram o avanço consistente das tarefas técnicas e documentais.

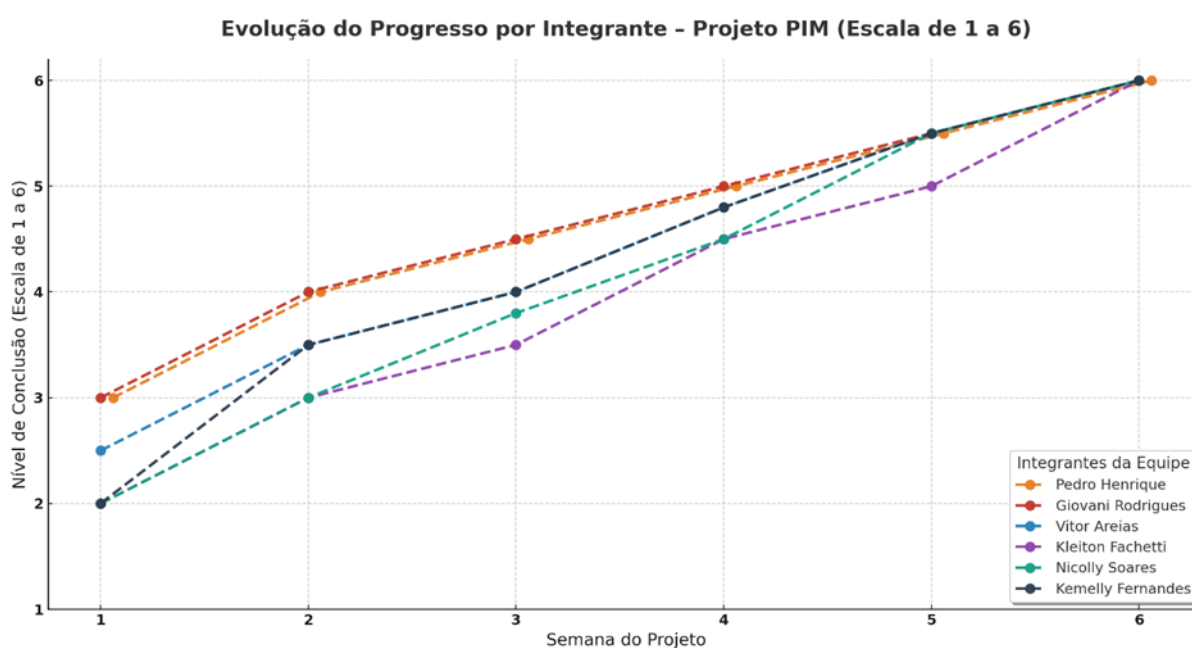
Entre os principais integrantes e suas contribuições, destacam-se:

- **Pedro Henrique de Paula Santos:** foco na modelagem UML, criação dos diagramas de caso de uso, sequência e classes, além da organização dos relatórios semanais e do acompanhamento do backlog no Jira. Também apoiou a documentação e as melhorias de interface.
- **Giovani Rodrigues de A. Reis:** responsável pela programação da interface em Python, integração dos módulos de aluno e professor e apoio na estruturação da lógica do sistema.
- **Vitor de Souza V. Areias:** principal desenvolvedor do sistema, responsável pela implementação do código-fonte, versão final do sistema aprimorado com auxílio de IA e manutenção do repositório oficial do projeto ([GitHub](#)).
- **Kleiton Fachetti:** colaborou na documentação técnica, diagrama de rede e nas revisões de código, garantindo consistência entre a UML e a implementação prática.
- **Nicolly Soares:** atuou na pesquisa de novas tecnologias, na testagem da interface e no aperfeiçoamento visual e funcional do sistema.
- **Kemelly Fernandes:** integrou a equipe com foco em apoio técnico, validação e testes funcionais, desempenhando papel relevante na verificação da integridade das funcionalidades implementadas e na otimização do fluxo de

trabalho no Jira. Também participou da fase de integração dos módulos e contribuiu para a análise de desempenho dos chatbots no ambiente do sistema.

O time foi responsável por executar as atividades de **análise, modelagem, codificação, testes e documentação**, de forma integrada e incremental. Essa divisão equilibrada de responsabilidades favoreceu a **autonomia coletiva e o aprendizado mútuo**, alinhando o grupo aos princípios centrais do Scrum.

Gráfico 1 – Progresso das Sprints da Equipe



Fonte: Autores

O gráfico apresentado ilustra a **evolução semanal do progresso individual dos integrantes do Projeto**, considerando uma **escala de conclusão de 1 a 6**, em que:

- **1** representa o início das atividades (planejamento e definição de papéis);
- **6** indica a **conclusão total das entregas**, com o sistema funcional, documentado e validado.

Cada linha tracejada representa um membro da equipe, demonstrando a **trajetória de desenvolvimento e contribuição** de cada integrante ao longo das **seis sprints** do projeto. Em síntese, este gráfico traduz visualmente a **sinergia do grupo, o**

domínio técnico adquirido e a eficácia do uso do Scrum na condução de um projeto acadêmico colaborativo.

A presença de Vitor como Scrum Master foi essencial para **organizar as atividades e manter o ritmo constante de evolução**, evidenciado pelo gráfico de progresso que demonstra um crescimento linear e estável das entregas de toda a equipe ao longo das seis semanas.

5 DIAGRAMA DE REDE DE COMPUTADORES

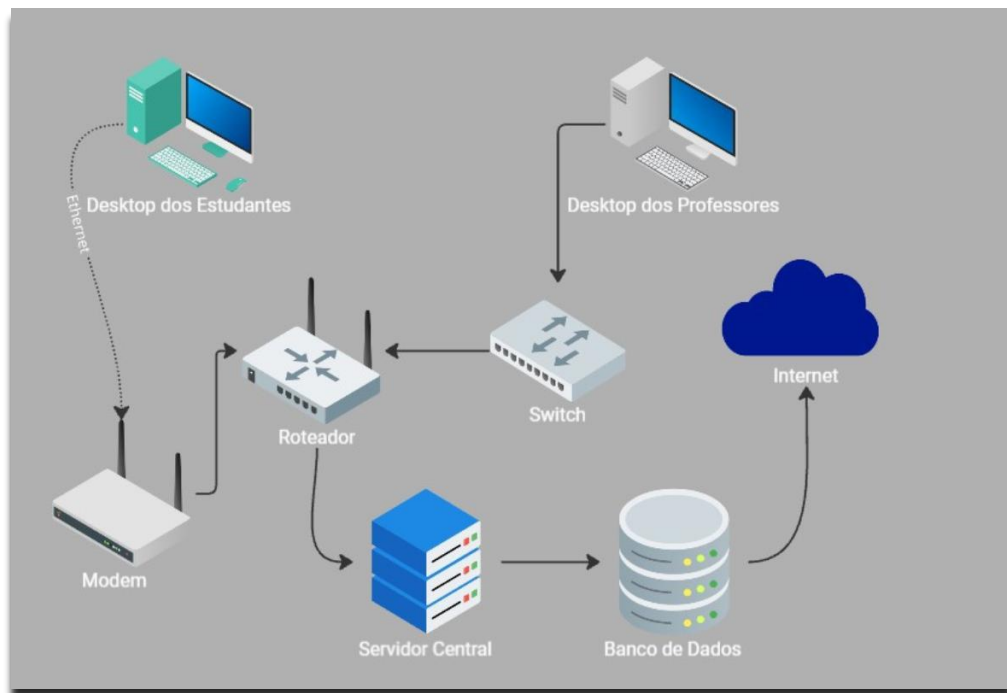
Um **diagrama de rede** é uma **representação gráfica** que mostra **como os dispositivos, servidores, serviços e usuários estão conectados dentro de um sistema computacional ou ambiente de rede**. Ele descreve a **topologia** (ou seja, a forma como os elementos se relaciona e troca dados) e o **fluxo de informações** entre os componentes que sustentam uma aplicação.

Em resumo, o **diagrama de rede** é o **mapa estrutural do sistema**. Ele demonstra **como a aplicação acadêmica funciona dentro da infraestrutura de rede**, assegurando que cada componente (usuário, servidor, banco de dados, chatbot e internet) esteja corretamente conectado e configurado para garantir:

- Segurança das informações;
- Eficiência na troca de dados;
- Escalabilidade e manutenção facilitada;
- Clareza na arquitetura do projeto.

Sem o diagrama de rede, o sistema ficaria **sem uma representação visual da sua base técnica**, o que dificultaria o entendimento, a manutenção e a justificativa de desempenho.

Figura 10 – Diagrama de Rede do Projeto



Fonte: Autores

O modelo proposto evidencia a interação entre os **usuários finais (alunos e professores)** e os **recursos computacionais centrais**, por meio de uma estrutura em camadas.

Componentes principais:

- **Desktop dos Estudantes:** estação de trabalho do usuário final, acessando o sistema via navegador conectado à rede interna (Ethernet).
- **Desktop dos Professores:** interface administrativa utilizada para lançar notas, cadastrar turmas e gerenciar alunos.
- **Modem e Roteador:** equipamentos responsáveis pela conexão com a internet e pelo roteamento do tráfego interno entre os dispositivos.
- **Switch:** dispositivo que gerencia o tráfego interno, garantindo comunicação eficiente entre servidores e terminais de acesso.
- **Servidor Central:** hospeda a aplicação desenvolvida em **Python (FastAPI)**, processando requisições, autenticação e regras de negócio.

- **Banco de Dados:** armazena as informações referentes a alunos, professores, turmas, notas e chatbots.
- **Internet (Nuvem):** permite o acesso remoto ao sistema e integra o ambiente local com o repositório GitHub e os módulos de inteligência artificial.

O diagrama de rede é fundamental para o **planejamento, segurança e manutenção** do sistema, pois:

- Facilita a **visualização da topologia de comunicação** entre os módulos.
- Permite a **identificação de possíveis gargalos** e pontos críticos de autenticação.
- Serve como **base técnica para manutenção e escalabilidade futura**.
- Contribui para a **documentação técnica completa do projeto**, sendo parte essencial da engenharia de software aplicada.

Em suma, o diagrama de rede garante **clareza estrutural, confiabilidade operacional e eficiência na integração dos componentes**, assegurando o desempenho contínuo do sistema acadêmico.

6 MANUAL DE USO DO SISTEMA

O Manual de Uso do Sistema Acadêmico Colaborativo com IA é o instrumento que assegura **padronização operacional, onboarding rápido e governança** do projeto, ao definir requisitos de acesso e execução, papéis de usuário e o fluxo completo das ações acadêmicas (criar turmas, cadastrar alunos, criar atividades, lançar notas e gerar relatórios). Ele também delimita o **escopo da IA** e sua utilidade prática no atendimento a dúvidas, orienta a leitura e interpretação dos **relatórios de desempenho** e fortalece a **sustentabilidade** pela digitalização de processos. Acesse o [GitHub](#) para ter uma visão geral de como o manual funciona e todos os processos do sistema.

O **Manual de Uso** exerce papel essencial, atuando como um **guia técnico e operacional** que assegura o uso correto e padronizado da aplicação. Ele esclarece o

funcionamento do sistema, detalha as funções de cada perfil de usuário e orienta sobre o fluxo de interação entre os módulos, garantindo clareza e eficiência no uso da plataforma.

6.1 EFICÁCIA DO MANUAL

O **Manual de Uso** é fundamental e para que todos compreendam:

- Servir como guia técnico para instalação, configuração e execução, [GitHub](#).
- Orientar novos usuários (professores e alunos) sobre o funcionamento do sistema.
- Garantir padronização do uso, evitando erros de operação.
- As responsabilidades de cada usuário.
- O papel da Inteligência Artificial no apoio ao aprendizado.
- E o impacto positivo da digitalização nos processos educacionais.

Fluxo resumido do sistema:

1. O **professor cria uma turma e cadastra os alunos.**
2. Ele **cria atividades e lança notas.**
3. O **aluno realiza entregas e consulta sua média.**
4. O **Chatbot responde dúvidas e o módulo de relatórios gera análises automáticas.**
5. Todos os dados são armazenados digitalmente e exibidos em tempo real.

6.2 MANUAL DO SISTEMA GARANTE

Especifica o **endereço de acesso à aplicação web**, os **requisitos técnicos mínimos** (como versão do Python, memória e sistema operacional compatível) e o **comando de inicialização do servidor local** (python main.py). Essas informações asseguram que qualquer usuário ou membro da equipe de desenvolvimento consiga

instalar, configurar e executar o sistema corretamente, sem necessidade de suporte adicional.

Perfis e Funcionalidades:

São descritos de forma clara os **papéis e permissões de cada tipo de usuário** — Aluno, Professor(a) e Chatbot (IA). O [documento](#) define o papel do professor(a) que pode **criar turmas, cadastrar alunos, gerenciar atividades, lançar e validar notas**, já o aluno **consulta média e status de aprovação**, além de poderem **utilizar o FAQ inteligente**. Essa organização garante controle de acesso, segurança das informações e delimitação precisa de responsabilidades dentro do ambiente acadêmico.

Figura 11 – Acesso do Aluno no Sistema

Campo	Valor
RA	H76D.JH0
Nome	Vitor Areias
Curso	ADS
Turma	01
NP1	10
NP2	10
PIM	10
Média	10

Fonte: Autores

Fluxo Operacional:

Apresenta, em formato de tabela e descrição sequencial, o **fluxo completo das ações do sistema**, processos como **criar turma, registrar aula** e corrigir

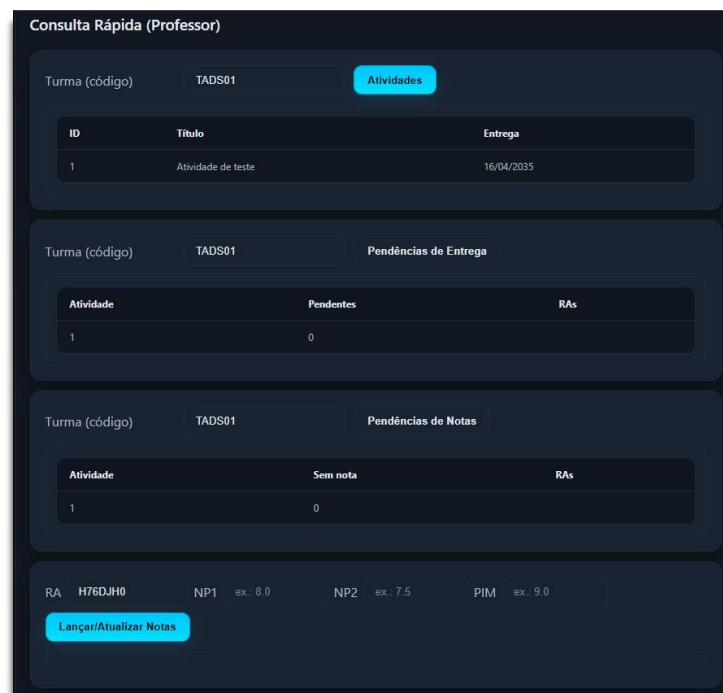
atividade. Essa estrutura padroniza a execução das tarefas e serve como guia para professores e alunos compreenderem o caminho ideal de uso do sistema.

Figura 12 – Acesso do Professor(a) no Sistema



Fonte: Autores

Figura 13 – Acesso Consulta Rápida e Funções do Professor(a)



Fonte: Autores

Integração com Inteligência Artificial:

O documento explica o funcionamento do **módulo de IA (Chatbot)**, responsável por **interpretar perguntas e fornecer respostas automáticas** com base em intenções textuais. Essa integração promove maior autonomia ao usuário, além

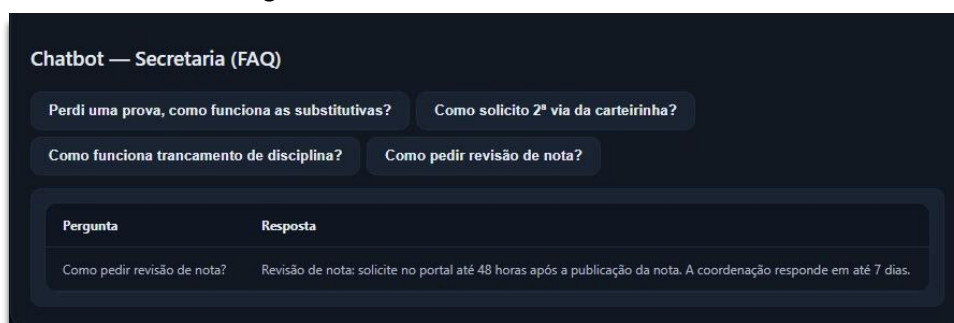
de facilitar o suporte pedagógico e técnico, permitindo que dúvidas frequentes sejam resolvidas sem intervenção humana direta.

Figura 14 – ChatBot do Professor(a)



Fonte: Autores

Figura 15 – ChatBot do Aluno



Fonte: Autores

Relatórios e Feedback:

O [manual](#) descreve as funções de **geração automática de relatórios** e de **feedback** de **desempenho**. Esses relatórios apresentam médias, distribuições de notas e panoramas gerais das turmas, permitindo que professores acompanhem a evolução acadêmica e adotem medidas de melhoria contínua. O histórico de atividades e avaliações também é armazenado e pode ser consultado a qualquer momento, promovendo transparência e rastreabilidade das informações.

Benefícios e Sustentabilidade:

O sistema documentado pelo manual contribui para a **eliminação de processos físicos**, substituindo o uso de papel por registros digitais. Além de tornar o ambiente acadêmico mais sustentável, essa digitalização otimiza o tempo, reduz erros de registro e centraliza as informações em um único sistema, alinhando-se às diretrizes de responsabilidade ambiental e eficiência tecnológica.

Interface Web:

O design adota um **tema escuro** e **layout em cards**, promovendo melhor legibilidade, contraste visual e uma experiência moderna e consistente em diferentes dispositivos.

A comunicação entre frontend e backend é feita via **fetch API (JavaScript)**, garantindo atualização em tempo real sem recarregar a página.

7 ARQUITETURA E ESTRUTURA TÉCNICA

A aplicação foi planejada seguindo princípios de **alta coesão e baixo acoplamento**, sendo dividida em módulos independentes dentro do diretório *app/*. Cada módulo é implementado como um *router* do FastAPI:

```

app/
├── main.py          # Ponto de entrada da API
├── routers/
│   ├── alunos.py    # CRUD e consultas de alunos
│   ├── turmas.py     # Gerenciamento de turmas
│   ├── atividades.py # Atividades e notas
│   ├── relatorios.py # Relatórios e médias
│   └── chatbot.py    # Chatbot (FAQ)
├── models.py        # Modelos Pydantic (validação de dados)
├── repositories.py  # Persistência local em JSON
├── templates/       # Interface HTML via Jinja2
└── static/          # Arquivos CSS e JS

```

A arquitetura modular garante **facilidade de manutenção e expansão futura**, como integração com bancos relacionais, dashboards e autenticação avançada.

Tecnologias e Ferramentas Técnicas Utilizadas:

- **Linguagem:** Python 3.13
- **Framework Web:** FastAPI
- **Servidor ASGI:** Uvicorn
- **Frontend:** HTML5, CSS3, JavaScript
- **Templating Engine:** Jinja2
- **Armazenamento Local:** JSON (data/db.json)
- **Bibliotecas Planejadas:** Chart.js (para gráficos)
- **Ambiente:** Virtualenv (venv)

O uso do **FastAPI** possibilita alta performance e documentação automática via **Swagger**. Já o **Pydantic** assegura integridade e tipagem estática dos dados trafegados entre cliente e servidor.

Chatbot Integrado (Inteligência Artificial)

O **Chatbot da Secretaria** é implementado em Python, com base em um dicionário de *intents* em JSON. Cada entrada define uma *key*, *pergunta* e *resposta*.

Por exemplo:

```
{  
  "key": "revisao_nota",  
  "pergunta": "Como pedir revisão de nota?",  
  "resposta": "Solicite no portal até 48h após a publicação. A coordenação responde em até 7 dias."  
}
```

Esse formato permite expansão fácil e futura substituição por um modelo NLP real, com *pattern matching* via expressões regulares e integração com serviços externos de IA.

Execução e Testes

Procedimento de execução local:

1. Ativar ambiente virtual:

```
.\.venv\Scripts\Activate.ps1
```

2. Instalar dependências:

```
pip install -r requirements.txt
```

3. Executar o servidor:

```
uvicorn app.main:app --reload
```

4. Acessar:

- Interface Web: <http://127.0.0.1:8000/ui>
- Documentação Swagger: <http://127.0.0.1:8000/docs>

Os testes envolveram simulações de CRUD (alunos/turmas), entregas de atividades, lançamento e revisão de notas, além da interação com o Chatbot.

A solução foi projetada para permitir expansão contínua, podendo futuramente incorporar **gráficos interativos (Chart.js)**, **banco de dados relacional**, **autenticação via JWT** e **pipelines de CI/CD** para implantação automatizada.

8 TÉCNOLOGIAS EMERGENTES

O sistema acadêmico colaborativo proposto tem como objetivo modernizar a gestão educacional, integrando professores, alunos e coordenadores em uma única plataforma inteligente, automatizada e sustentável. A aplicação de tecnologias emergentes é essencial para alcançar esses objetivos, proporcionando inovação, segurança, eficiência e uma experiência de uso mais dinâmica e personalizada. A

seguir, são descritas as principais tecnologias e como elas podem ser aplicadas de forma prática no desenvolvimento do sistema.

A **Inteligência Artificial (IA)** representa o núcleo do sistema proposto, atuando em múltiplas frentes. Um exemplo é o **chatbot acadêmico inteligente**, alimentado por modelos de linguagem natural, capaz de interagir com alunos e professores em tempo real. Essa IA pode responder dúvidas sobre prazos, notas, faltas, disciplinas e até emitir alertas automáticos sobre pendências acadêmicas. Além disso, a IA pode auxiliar na geração de relatórios automatizados e no suporte técnico do sistema, otimizando o tempo de atendimento e reduzindo a sobrecarga das secretarias. Conforme Marr (2019), o uso de IA generativa em sistemas corporativos melhora significativamente a eficiência operacional e a experiência do usuário.

Outra aplicação relevante é o uso de **agentes autônomos de IA (AI Agents)**, que executam tarefas administrativas sem intervenção humana. Esses agentes podem, por exemplo, enviar lembretes automáticos sobre atividades a serem entregues, avisos de reuniões ou relatórios periódicos sobre o desempenho dos alunos. Essa automação torna o sistema mais proativo e diminui a necessidade de acompanhamento manual, garantindo que informações importantes sejam comunicadas de forma automática e personalizada.

O sistema também poderá incluir **módulos de aprendizado de máquina (Machine Learning)** para realizar **análises preditivas**. Por meio de algoritmos de regressão e classificação, é possível identificar padrões de comportamento e prever o desempenho dos alunos, destacando aqueles que estão em risco de reprovação ou evasão. Isso permite a criação de alertas preventivos e estratégias pedagógicas personalizadas. Segundo Goodfellow, Bengio e Courville (2016), a aprendizagem de máquina é uma ferramenta poderosa para transformar dados em conhecimento e apoiar decisões baseadas em evidências.

Em termos de infraestrutura, o projeto se beneficia da **computação em nuvem** e da **arquitetura de microsserviços**, que garantem escalabilidade, flexibilidade e maior segurança ao sistema. A hospedagem em plataformas como AWS, Azure ou Google Cloud permite que o sistema suporte múltiplos usuários simultâneos,

mantendo a estabilidade e a performance. A arquitetura em microsserviços facilita a manutenção e a expansão futura do sistema, possibilitando o desenvolvimento modular de diferentes componentes, como o portal do aluno, o painel do professor e o módulo de relatórios administrativos (TANENBAUM; WETHERALL, 2011).

Para garantir integração e conectividade, o sistema pode utilizar **APIs educacionais abertas (Open Education APIs)**, que permitem a comunicação entre diferentes plataformas acadêmicas, como bibliotecas digitais, ambientes virtuais de aprendizagem (AVA) e sistemas de gestão de cursos. Essa interoperabilidade oferece ao usuário uma experiência unificada, centralizando informações de diversas fontes em um único painel de controle. Isso simplifica a rotina acadêmica e melhora a eficiência no acesso às informações.

A segurança e a transparência dos dados acadêmicos podem ser fortalecidas por meio da **tecnologia blockchain**, utilizada para registrar e validar notas, certificados e históricos escolares de forma imutável. Essa inovação elimina riscos de adulteração de registros e garante a autenticidade das informações acadêmicas. Cada registro de nota ou certificado seria tratado como um bloco único, validado na cadeia e acessível apenas a usuários autorizados. De acordo com Russell e Norvig (2021), o blockchain promove integridade e rastreabilidade, características essenciais em sistemas que lidam com dados sensíveis.

A **interface do sistema** é outro ponto de inovação, sendo projetada com base em **design adaptativo e inteligência de contexto**. Utilizando ferramentas como React, Tailwind e Framer Motion, a interface se ajusta conforme o tipo de usuário logado — exibindo, por exemplo, notas e presenças para alunos, ou relatórios de desempenho e lançamento de notas para professores. Essa personalização aumenta a usabilidade e cria uma experiência mais agradável e intuitiva. Além disso, pode-se incorporar uma **IA de interface**, que fornece recomendações automáticas, como “Você tem uma atividade pendente para corrigir” ou “Há novos materiais disponíveis para sua turma”.

Outro aspecto essencial do sistema é a **sustentabilidade digital**, alinhada às diretrizes de responsabilidade ambiental. O sistema promove a redução do uso de

papel, permitindo que relatórios, provas e documentos sejam emitidos e armazenados digitalmente. Também é possível incluir um **painel de sustentabilidade**, que exibe indicadores sobre a economia de papel e o impacto ambiental positivo gerado pela digitalização de processos. Essa iniciativa reforça o compromisso institucional com a educação ambiental e as práticas sustentáveis.

Em síntese, o **Sistema Acadêmico Colaborativo com Apoio de IA** integra múltiplas tecnologias emergentes — inteligência artificial, aprendizado de máquina, blockchain, microsserviços, APIs e design inteligente — para criar uma plataforma completa, moderna e segura. Essa combinação tecnológica transforma o sistema acadêmico tradicional em uma solução inovadora, colaborativa e sustentável, que beneficia tanto os gestores quanto a comunidade acadêmica. Assim, contribui para a transformação digital no ambiente educacional, promovendo uma gestão acadêmica mais eficiente, integrada e orientada por dados.

9 CONSIDERAÇÕES FINAIS

O desenvolvimento do Sistema Acadêmico Colaborativo com Apoio de Inteligência Artificial (IA) constituiu uma experiência acadêmica significativa, consolidando conhecimentos teóricos e práticos de engenharia de software, algoritmos e estruturas de dados em Python, programação estruturada em C, modelagem UML, redes de computadores, metodologias ágeis, inteligência artificial, educação ambiental e tecnologias emergentes. A integração desses conteúdos resultou em uma solução tecnológica robusta, escalável e alinhada às demandas contemporâneas da gestão educacional digital.

A proposta atendeu plenamente ao objetivo geral, ao projetar e implementar um sistema colaborativo voltado à centralização da comunicação e da administração acadêmica entre alunos, professores e secretaria. A aplicação de práticas de metodologia ágil (Scrum) viabilizou um processo de desenvolvimento iterativo e colaborativo, com entregas contínuas e de valor agregado, utilizando ferramentas como o Atlassian Jira Software para acompanhamento das sprints, priorização de tarefas e controle de progresso.

Do ponto de vista técnico, a utilização do framework FastAPI em conjunto com a linguagem Python garantiu alta performance, modularidade e segurança nas operações do sistema. O emprego de diagramas UML (casos de uso, classes e sequência) proporcionou clareza na modelagem do sistema e na comunicação entre os integrantes da equipe, assegurando coerência entre o planejamento e a implementação. O diagrama de rede projetado evidenciou a viabilidade da aplicação em um ambiente cliente-servidor, reforçando os conceitos de redes locais e sistemas distribuídos exigidos no escopo do projeto.

Um dos principais diferenciais do sistema foi a implementação do chatbot com Inteligência Artificial, que atua como assistente virtual acadêmico, respondendo a dúvidas frequentes e automatizando atendimentos, demonstrando na prática o potencial da IA aplicada à educação. Essa funcionalidade reforçou o caráter inovador e sustentável do projeto, reduzindo a sobrecarga da secretaria, otimizando o tempo de resposta aos alunos e eliminando processos manuais e impressos, em consonância com os princípios de educação ambiental e digitalização consciente.

A equipe superou desafios técnicos e conceituais por meio da cooperação, da aplicação de boas práticas de desenvolvimento e da comunicação contínua entre os membros, fortalecendo competências interpessoais, como liderança, planejamento e trabalho em equipe. O uso de repositórios GitHub para controle de versões e documentação favoreceu a rastreabilidade e a transparência do código, além de garantir a integridade das entregas.

Em síntese, o Sistema Acadêmico Colaborativo com Apoio de IA representa mais do que uma aplicação tecnológica: trata-se de uma solução inovadora, sustentável e colaborativa, que transforma a rotina acadêmica, promove a integração entre os agentes educacionais e reforça o compromisso da universidade com a inovação, responsabilidade ambiental e formação de profissionais preparados para os desafios do futuro digital. O projeto consolida a interdisciplinaridade como ferramenta essencial no desenvolvimento de sistemas modernos, eficientes e voltados à melhoria contínua da educação.

10 CONCLUSÃO

O **Sistema Acadêmico Colaborativo com Apoio de Inteligência Artificial (IA)** concretizou-se como uma proposta inovadora e multidisciplinar, capaz de integrar conceitos teóricos e práticos do curso de **Análise e Desenvolvimento de Sistemas**, unindo tecnologia, metodologias ágeis e responsabilidade social. O projeto demonstrou que a combinação entre **engenharia de software, inteligência artificial e redes de computadores** pode gerar soluções reais e eficazes para problemas de gestão acadêmica e comunicação institucional.

Durante o desenvolvimento, foi possível aplicar, de forma integrada, os conhecimentos adquiridos em **Engenharia de Software Ágil, Python, Programação Estruturada em C, Análise e Projeto de Sistemas, Redes, IA e Educação Ambiental**, comprovando a importância da interdisciplinaridade. A utilização do **framework FastAPI** e da **linguagem Python** proporcionou uma arquitetura moderna, segura e modular, enquanto a aplicação do **Scrum** garantiu organização, colaboração e entregas contínuas de valor.

A **implementação do chatbot com IA** se destacou como um dos principais diferenciais do sistema, trazendo uma experiência de atendimento automatizada, acessível e dinâmica. Essa inovação reforça o papel da IA como ferramenta de apoio à educação, simplificando o acesso à informação e ampliando a autonomia dos usuários. O sistema também apresentou uma **interface intuitiva**, integração de **módulos de relatórios automáticos** e um **fluxo otimizado de comunicação entre professores, alunos e secretaria**, evidenciando o potencial da automação no contexto educacional.

Além dos resultados técnicos, o projeto promoveu **aprendizado coletivo, disciplina e cooperação** entre os integrantes do grupo, que atuaram de forma colaborativa na resolução de desafios e na consolidação das entregas. Essa vivência prática contribuiu significativamente para o amadurecimento profissional dos participantes, fortalecendo competências fundamentais como planejamento, pensamento crítico, gestão de tempo e trabalho em equipe — pilares indispensáveis ao mercado de tecnologia.

Outro aspecto relevante foi a **consciência ambiental e social** incorporada ao projeto. A substituição de processos físicos por soluções digitais reflete o compromisso com a sustentabilidade, reduzindo o consumo de papel e promovendo uma cultura de responsabilidade ambiental.

Portanto, o **Sistema Acadêmico Colaborativo com Apoio de IA** cumpre plenamente os objetivos propostos no PIM II, apresentando-se como uma solução tecnológica moderna, funcional e escalável, que pode ser expandida para futuras aplicações com novas camadas de IA, banco de dados relacionais e aprendizado de máquina. Mais do que um produto técnico, este trabalho representa um **marco de aprendizado prático e inovação acadêmica**, consolidando o papel do aluno como agente transformador no processo de digitalização e modernização da educação.

Em síntese, o projeto reforça que **a integração entre conhecimento, tecnologia e propósito social** é o caminho para o desenvolvimento de soluções inteligentes, sustentáveis e humanizadas — um reflexo do compromisso da **Universidade Paulista (UNIP)** em formar profissionais éticos, criativos e preparados para os desafios do futuro digital.

REFERÊNCIAS

OBJECT MANAGEMENT GROUP (OMG). **ABOUT THE UNIFIED MODELING LANGUAGE (UML) SPECIFICATION VERSION 2.5.1.** DISPONÍVEL EM: <HTTPS://WWW.OMG.ORG/SPEC/UML/2.5.1/ABOUT-UML>. ACESSO EM: 16 OUT. 2025.

OBJECT MANAGEMENT GROUP (OMG). **WHAT IS THE UNIFIED MODELING LANGUAGE (UML)?** DISPONÍVEL EM: <HTTPS://WWW.OMG.ORG/UML/WHAT-IS-UML.HTM>. ACESSO EM: 16 OUT. 2025.

UML-DIAGRAMS.ORG. **CLASS DIAGRAMS – UML-DIAGRAMS.ORG.** DISPONÍVEL EM: <HTTPS://WWW.UML-DIAGRAMS.ORG/CLASS-DIAGRAMS.HTML>. ACESSO EM: 16 OUT. 2025.

Visual Paradigm Guide to UML. **“What is Unified Modeling Language (UML)?”.** Disponível em: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-uml/>. Acesso em: 16 out. 2025.

Miro Blog. **«What is UML Diagram? | Different Types and Benefits».** Disponível em: <https://miro.com/diagramming/what-is-a-uml-diagram/>. Acesso em: 16 out. 2025.

Lucidchart Blog. **“What are the types of UML diagrams?”.** Disponível em: <https://www.lucidchart.com/blog/types-of-UML-diagrams>. Acesso em: 16 out. 2025.

Tutorialspoint. **UML – Overview.** Disponível em: https://www.tutorialspoint.com/uml/uml_overview.htm. Acesso em: 16 out. 2025.

IBM DEVELOPER. **Explore the UML sequence diagram.** Disponível em: <https://developer.ibm.com/articles/the-sequence-diagram/>. Acesso em: 23 out. 2025.

FOWLER, Martin. **UML Distilled. 3. ed. Boston: Addison-Wesley, 2004.** Disponível em: <https://martinfowler.com/books/uml.html>. Acesso em: 23 out. 2025.

LARMAN, Craig. **Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. 3. ed. Upper Saddle River: Prentice Hall, 2004.** (Página da O'Reilly). Disponível em: <https://www.oreilly.com/library/view/applying-uml-and/0131489062/>. Acesso em: 23 out. 2025.

SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide: The Definitive Guide to Scrum.** Scrum.org, 2020.

ATLASSIAN. **Jira Software Documentation.** Sydney: Atlassian Corporation Plc, 2024. Disponível em: <https://support.atlassian.com/jira-software/>. Acesso em: 7 out. 2025.

FASTAPI. **FastAPI Framework – High Performance, Easy to Learn, Fast to Code, Ready for Production.** Disponível em: <https://fastapi.tiangolo.com>. Acesso em: 11 ago. 2025.

GITHUB. **Repositório Oficial do Projeto PIM II – Sistema Acadêmico Colaborativo com Inteligência Artificial**. 2025. Disponível em: <https://github.com/>. Acesso em: 9 ago. 2025.

UNIVERSIDADE PAULISTA – UNIP. **Projeto Integrado Multidisciplinar – PIM II: Sistema Acadêmico Colaborativo para Ambiente Educacional com Suporte de Chatbot e Inteligência Artificial**. São Paulo: UNIP, 2025. Documento acadêmico interno.

ISO/IEC. **Software Engineering — Software Life Cycle Processes. ISO/IEC 12207:2017**. Geneva: International Organization for Standardization, 2017.

SCRUM ALLIANCE. **Scrum Guide 2020: The Definitive Guide to Scrum — The Rules of the Game. Ken Schwaber & Jeff Sutherland**. Disponível em: <https://scrumguides.org>. Acesso em: 7 out. 2025.

TANENBAUM, Andrew S.; WETHERALL, David J. **Redes de Computadores**. 6. ed. Rio de Janeiro: Pearson, 2021.

PYTHON SOFTWARE FOUNDATION. **Python 3.10 Documentation**. Disponível em: <https://docs.python.org/3/>. Acesso em: 18 ago. 2025.

FACHETTI, Kleiton. **PIM-II-Projeto-Academico. GitHub repository**. Disponível em: <https://github.com/KleitonFachetti/PIM-II-Projeto-Academico.git>. Acesso em: 28 out. 2025.

UVICORN. **Uvicorn ASGI Server Documentation**. Disponível em: <https://www.uvicorn.org>. Acesso em: 14 out. 2025.

PYDANTIC. **Pydantic User Guide**. Disponível em: <https://docs.pydantic.dev>. Acesso em: 14 out. 2025.

LANGCHAIN. **LangChain Documentation**. Disponível em: <https://www.langchain.com/>. Acesso em: 28 out. 2025.

MARR, Bernard. **Artificial Intelligence in Practice: How 50 Successful Companies Used AI and Machine Learning to Solve Problems**. Hoboken: Wiley, 2019.

RUSSELL, Stuart; NORVIG, Peter. **Artificial Intelligence: A Modern Approach**. 4. ed. Boston: Pearson, 2021.

ISO/IEC. **Information Technology – Artificial Intelligence – Concepts and Terminology (ISO/IEC 22989:2022)**. Geneva: International Organization for Standardization, 2022.

GOOGLE CLOUD. **AI and Machine Learning Products**. Disponível em: <https://cloud.google.com/products/ai>. Acesso em: 28 out. 2025.

MICROSOFT AZURE. **AI and Machine Learning Services**. Disponível em: <https://azure.microsoft.com/en-us/products/ai-services/>. Acesso em: 28 out. 2025.

ETHEREUM FOUNDATION. **Ethereum Whitepaper.** Disponível em:
<https://ethereum.org/en/whitepaper/>. Acesso em: 12 out. 2025.

HYPERLEDGER FOUNDATION. **Hyperledger Overview.** Disponível em:
<https://www.hyperledger.org/>. Acesso em: 12 out. 2025.

GRAFANA LABS. **Grafana Documentation.** Disponível em:
<https://grafana.com/docs/>. Acesso em: 14 out. 2025.

POWER BI. **Documentação Oficial Microsoft Power BI.** Disponível em:
<https://learn.microsoft.com/power-bi/>. Acesso em: 14 out. 2025.